BAB II
TINJAUAN PUSTAKA

Bab ini membahas penelitian dan konsep yang mendukung pengembangan sistem pemeriksaan keamanan kode hasil LLM. Pembahasan dimulai dari keamanan kode yang dihasilkan LLM dan dataset yang digunakan untuk mengujinya. Selanjutnya dibahas pemeriksaan kode dengan SAST, kerja sama beberapa agen, perbaikan kode, serta serangan yang dapat memengaruhi hasil pemeriksaan. Pembahasan dibatasi pada konsep dan hasil penelitian yang berkaitan dengan pemeriksaan kode hasil LLM.

2.1 Keamanan Kode Hasil LLM

2.1.1 Kebenaran dan keamanan kode

Large language model (LLM) dapat digunakan untuk membuat kode dari instruksi bahasa alami. Meskipun kode tersebut dapat dijalankan dan memberikan hasil yang benar, kode belum tentu aman. Sebagai contoh, program dapat menampilkan data sesuai permintaan pengguna, tetapi tidak memeriksa apakah pengguna berhak mengakses data tersebut. Oleh karena itu, keberhasilan program menjalankan tugas dan keamanan program perlu diperiksa secara terpisah.

Untuk mengelompokkan kelemahan kode, dataset yang dibahas pada bab ini menggunakan *Common Weakness Enumeration* (CWE). Setiap CWE menunjukkan suatu jenis kelemahan, seperti pemeriksaan masukan yang tidak memadai atau penggunaan kata sandi di dalam kode. Pengelompokan ini membantu peneliti menentukan jenis kelemahan yang ingin diuji dan membandingkan temuan pada jenis yang sama (Siddiq & Santos, 2022; Tony et al., 2023). Namun, satu program dapat memiliki lebih dari satu kelemahan. Pemeriksaan terhadap satu CWE belum mencakup seluruh masalah keamanan pada program tersebut.

CyberSecEval menilai keamanan LLM dari dua sisi, yaitu kecenderungan menghasilkan kode rentan dan kesediaan membantu serangan siber. Pada pengujian keluarga Llama 2, Code Llama, dan GPT, sekitar 30% saran kode pada benchmark

tersebut dilaporkan rentan. Alat pendeteksi yang digunakan memperoleh presisi 96% dan *recall* 79% berdasarkan pemeriksaan manual (Bhatt et al., 2023). Presisi menunjukkan ketepatan temuan, sedangkan recall menunjukkan seberapa banyak kelemahan yang berhasil ditemukan. Angka kemampuan alat pendeteksi tersebut berbeda dari persentase kode rentan yang dihasilkan model.

Hasil penelitian itu juga menunjukkan bahwa kemampuan membuat kode yang lebih baik tidak selalu diikuti oleh keamanan yang lebih baik. Hal serupa ditemukan pada perbandingan tujuh alat LLM oleh Morkonda dkk. Penelitian tersebut menggunakan 81 prompt Python dari LLMSecEval yang mencakup 12 kategori CWE. Alat yang diuji adalah GPT-4o, Claude 3 Sonnet, Gemini, Perplexity AI, Phind-70b, CodeWhisperer, dan watsonx. Dengan pemeriksaan CodeQL dan GPT-4o untuk kategori tertentu, sedikitnya sekitar 73% potongan kode pada setiap alat dinilai memiliki kerentanan (Morkonda et al., 2026). Angka ini berlaku pada tugas yang dipilih dalam penelitian tersebut, sehingga tidak menggambarkan seluruh penggunaan LLM.

2.1.2 Pengaruh prompt dan cara pengujian

Penelitian keamanan kode umumnya menggunakan prompt yang disiapkan untuk percobaan atau mengambil kode dari penggunaan nyata. Prompt yang disiapkan peneliti memudahkan pengujian karena tugas yang diberikan kepada setiap model dapat dibuat sama. Namun, bentuk tugas tersebut dapat berbeda dari permintaan pengguna dalam kegiatan pengembangan perangkat lunak.

Penelitian pada dataset DevGPT menggunakan 2.315 potongan kode C, C++, dan C# dari percakapan pengembang dengan LLM. Pemindaian kode dan pemeriksaan manual menemukan 56 kerentanan pada 48 berkas. Penelitian tersebut juga menguji GPT-4.1, GPT-5, dan Claude Opus 4.1 untuk mendeteksi serta memperbaiki kelemahan (Belozerov et al., 2026). Berbeda dari pengujian menggunakan prompt keamanan, data DevGPT mencakup kode yang diberikan pengguna dan kode yang dibuat model. Karena sumber data dan cara pemeriksaannya berbeda, hasilnya tidak dapat langsung dibandingkan dengan persentase kerentanan pada LLMSecEval.

WildCode Revisited juga membahas keamanan kode dari penggunaan ChatGPT yang nyata. Penelitian tersebut menunjukkan bahwa perhatian pengguna terhadap keamanan masih terbatas (Khanmohammadi et al., 2026). Sementara itu, Kharma dkk. menemukan bahwa perubahan cara menyusun prompt dapat mengubah jenis kelemahan yang muncul, tetapi belum memberikan penurunan jumlah atau kepadatan kerentanan yang signifikan secara statistik pada pengujian mereka (Kharma et al., 2026). Dengan demikian, permintaan untuk membuat kode aman tetap perlu diikuti pemeriksaan kode. Hasil pada bahasa lain, seperti C/C++ dalam LLM-CSEC, juga perlu disesuaikan sebelum digunakan untuk menjelaskan keamanan kode Python (Shahid et al., 2025).

Cara menyusun benchmark turut memengaruhi hasil pengujian. Hariharan dkk. menunjukkan bahwa komentar, nama pengenal, dan instruksi pada CyberSecEval dapat memberi petunjuk yang mendorong model menghasilkan pola tidak aman. Ketika petunjuk tersebut diubah, jumlah keluaran yang dinilai aman oleh alat pendeteksi juga berubah (Hariharan et al., 2024). Selain itu, dataset publik mungkin telah muncul dalam data pelatihan model. Hal ini perlu diperhatikan agar kemampuan pada dataset lama tidak langsung dianggap mewakili kemampuan pada kode yang benar-benar baru (Chen et al., 2025).

2.1.3 Dataset penelitian

SecurityEval menyediakan prompt dan contoh kode Python rentan yang dikelompokkan berdasarkan CWE. Pemeriksaan `dataset.jsonl` pada versi repositori `6f4fb70` menghasilkan 121 sampel dengan 69 kategori CWE berdasarkan kolom `ID`. LLMSecEval pada versi `39e97f0` berisi 150 baris tugas dan 18 kategori CWE. Sebanyak 83 baris berlabel Python dan mencakup 12 kategori CWE (Siddiq & Santos, 2022; Tony et al., 2023). Jumlah tersebut menunjukkan isi snapshot dataset, bukan jumlah percobaan penelitian ini.

Kedua dataset menyediakan bentuk tugas yang berbeda, yaitu pelengkapan kode dan pembuatan kode dari instruksi bahasa alami. Keduanya memiliki 12 kategori CWE Python yang sama. Pada LLMSecEval, 83 baris Python menggunakan 81 `Prompt ID` unik karena dua ID dipakai masing-masing dua kali.

Isi prompt dan nama berkas pada baris tersebut berbeda. Jumlah 81 ID ini sama dengan jumlah yang dilaporkan Morkonda dkk., tetapi kesamaan hitungan belum membuktikan aturan seleksi yang digunakan penulis. Oleh karena itu, baris dataset dan ID perlu dibedakan agar tugas yang berbeda tidak dihapus hanya karena ID-nya sama (Morkonda et al., 2026; Tony et al., 2023).

CodeSecEval menyediakan pembanding yang menggabungkan pengujian fungsi dan keamanan. Versi tahun 2024 memuat 180 sampel dan 44 jenis kerentanan, sedangkan revisi September 2026 memuat 255 tugas Python dengan 77 CWE (Wang et al., 2024; Wang et al., 2026a). Revisi tersebut memperkenalkan SecAwareCoder yang menghubungkan pemodelan ancaman sesuai tugas, pembuatan kode dengan batasan keamanan, pembuatan pengujian, dan perbaikan berdasarkan hasil eksekusi. Dataset revisi telah tersedia di Hugging Face. Namun, pada pemeriksaan 20 Oktober 2026, repositori resmi menyatakan bahwa kode SecAwareCoder belum dirilis. Pembandingan metode karena itu dibedakan dari replikasi implementasinya.

2.2 Pemeriksaan Kerentanan dengan SAST dan LLM

2.2.1 Pemeriksaan kode secara statis

Static application security testing (SAST) adalah pemeriksaan keamanan kode tanpa menjalankan program. Pemeriksaan dapat dilakukan dengan mencari pola tertentu, memeriksa struktur kode, atau mengikuti aliran data. Sebagai contoh, alat dapat memeriksa apakah masukan pengguna dipakai dalam perintah basis data tanpa penanganan yang sesuai. Pendekatan ini memungkinkan masalah ditemukan sebelum program digunakan.

Meskipun demikian, hasil SAST masih dapat keliru. Alat dapat memberi peringatan pada kode yang sebenarnya tidak rentan, yang disebut *false positive*. Sebaliknya, kelemahan yang tidak ditemukan disebut *false negative*. Kesalahan dapat terjadi ketika alat tidak memahami fungsi pustaka, pemeriksaan masukan buatan pengembang, atau hubungan antarbagian program (Iranmanesh et al., 2025; Li et al., 2025a). Oleh karena itu, peringatan dari alat perlu diperiksa bersama konteks kode.

Setiap alat juga memiliki cakupan aturan yang berbeda. Kode yang tidak diberi peringatan oleh Bandit masih mungkin ditandai oleh CodeQL, sebagaimana dibahas dalam Multi-LLMSecCodeEval (Sabir et al., 2026). Pada penelitian ini, penggunaan Semgrep dan Bandit perlu disertai catatan versi serta aturan yang digunakan. Hasil tanpa peringatan juga perlu dibedakan dari kegagalan membaca kode atau tidak tersedianya aturan untuk suatu CWE. Ketiga keadaan tersebut memiliki arti yang berbeda bagi keputusan gate.

2.2.2 Penggunaan LLM untuk memeriksa hasil SAST

LLM dapat digunakan untuk membantu memahami konteks yang sulit ditangani aturan statis. Salah satu bentuknya adalah pendekatan *neuro-symbolic*, yaitu penggabungan kemampuan model dalam memahami kode dengan analisis berbasis aturan. IRIS menggunakan LLM untuk membantu menentukan sumber data, bagian kode yang menerima data, dan konteks pemakaiannya dalam CodeQL. Pada versi ICLR 2025, IRIS dengan GPT-4 mendeteksi 55 dari 120 kerentanan dalam CWE-Bench-Java, dibandingkan 27 yang ditemukan CodeQL. Rata-rata *false discovery rate*, yaitu proporsi temuan yang dinilai salah menurut ukuran penelitian tersebut, turun dari 90,03% menjadi 84,82% (Li et al., 2025a).

LLM juga dapat diberi tugas yang lebih terbatas, yaitu memeriksa peringatan yang sudah ditemukan alat. Proses ini disebut triase. ZeroFalse menyediakan jejak aliran data, konteks kode, dan informasi CWE kepada LLM untuk membantu menentukan apakah suatu peringatan perlu dipertahankan. Pengujiannya mencakup OWASP Java Benchmark dan proyek nyata (Iranmanesh et al., 2025). SAST-Genius menggabungkan Semgrep 1.97.0 dengan Llama 3 8B yang telah di-fine-tune. Pengujiannya memakai 25 repositori Python, Java, dan JavaScript, dengan 170 kerentanan acuan. Dalam laporan penulis, false positive turun dari 225 menjadi 20 (Agrawal & Ahi, 2025). Angka tersebut berasal dari pengujian campuran bahasa dan tidak langsung menunjukkan besarnya peningkatan pada kode Python hasil LLM.

QASecClaw menggunakan pembagian tugas yang serupa. Semgrep menghasilkan temuan awal, lalu agen filter berbasis Qwen 3.5 Plus memeriksa temuan tersebut bersama kode. Pada 2.740 kasus Java dalam OWASP Benchmark

v1.2, jumlah false positive turun dari 560 menjadi 64, tetapi recall juga sedikit menurun. Sistemnya menyediakan bagian untuk menggabungkan bukti pengujian dinamis, tetapi bagian tersebut tidak dijalankan pada percobaan benchmark statis (Ameen et al., 2026). Dengan demikian, keberadaan komponen pengujian dalam rancangan belum berarti seluruh temuan telah dibuktikan melalui eksekusi kode.

Berbeda dari penyaringan peringatan, QRS menggunakan LLM untuk membentuk kueri CodeQL dan memeriksa temuannya pada paket Python utuh (Tsigkourakos & Patsakis, 2026). Paket utuh menyediakan konteks yang lebih lengkap daripada sebuah fungsi atau potongan kode. Perbedaan ini perlu diperhatikan ketika cara kerja QRS diterapkan pada kode hasil LLM. Sebagian besar penelitian tersebut memakai kode proyek atau benchmark yang telah tersedia, sehingga hasilnya masih perlu diuji pada kode baru yang dibuat model.

2.2.3 Peran SAST dalam pengambilan keputusan

Dalam rancangan penelitian ini, agen Analyse memeriksa kode dengan bantuan Semgrep dan Bandit. Agen Triage kemudian menilai apakah temuan tersebut sesuai dengan kondisi kode. Pemisahan tugas ini diperlukan karena menemukan pola yang mencurigakan dan memutuskan kebenaran suatu temuan merupakan dua pekerjaan berbeda. Hasil pemeriksaan perlu memuat lokasi, jenis CWE, tingkat keparahan, aturan asal, dan alasan yang mendukung temuan.

Jika beberapa alat melaporkan masalah yang sama, temuan perlu digabungkan agar satu kelemahan tidak dihitung berulang. Setiap hasil juga perlu dihubungkan dengan versi kode yang diperiksa. Setelah kode diperbaiki, hasil pemeriksaan versi sebelumnya tidak dapat langsung digunakan untuk menerima versi baru. Penjelasan LLM dapat membantu memahami masalah, tetapi tetap perlu dibedakan dari bukti hasil pengujian.

Penilaian triase perlu dibedakan dari keputusan penerimaan kode. QASecClaw menunjukkan bahwa penyaringan peringatan dapat menurunkan false positive sekaligus mengurangi recall (Ameen et al., 2026). Karena itu, penolakan suatu temuan perlu disertai alasan dan bukti yang dapat ditelusuri. Aturan tingkat

keparahan dan tindak lanjut merupakan bagian dari rancangan gate yang perlu dinilai dalam percobaan.

2.3 Sistem Multi-Agen untuk Keamanan Kode

2.3.1 Pembagian tugas antaragen

Sistem multi-agen membagi pekerjaan kepada beberapa agen yang saling bertukar informasi. Dalam pemeriksaan keamanan kode, agen dapat bertugas membuat kode, mencari kelemahan, menguji temuan, atau memperbaiki kode. Pembagian ini membantu setiap agen berfokus pada tugas tertentu. Namun, manfaatnya tetap bergantung pada cara agen bekerja sama dan bagaimana hasilnya diperiksa.

Tujuan sistem perlu dibedakan sebelum hasil penelitian dibandingkan. CodeAgent digunakan untuk membantu proses peninjauan kode, sedangkan VulTrial dan MAVUL lebih berfokus pada pemeriksaan kerentanan pada kode yang sudah tersedia (Li et al., 2025b; Tang et al., 2024; Widyasari et al., 2025). Sistem yang berhasil mengenali kelemahan belum tentu sudah mengatur pembuatan kode, perbaikannya, dan keputusan apakah kode dapat diterima. Perbedaan tugas ini menjadi dasar dalam memilih pembanding untuk penelitian ini.

Beberapa agen dapat menggunakan model yang sama dengan instruksi berbeda. Pilihan lainnya adalah menggunakan model yang berbeda untuk setiap tugas. Instruksi yang berbeda dapat mengarahkan perhatian agen pada bagian kode yang berbeda, tetapi tidak menjamin pendapatnya saling bebas. Jika agen memiliki kelemahan pemahaman yang sama, beberapa agen dapat menghasilkan kesalahan yang sama pula. Karena itu, manfaat penambahan agen perlu dilihat dari perubahan hasil pemeriksaan, bukan dari banyaknya pendapat yang dihasilkan.

2.3.2 Pembuatan, pemeriksaan, dan pengujian kode

AutoSafeCoder menggabungkan tiga agen, yaitu Coding Agent, Static Analyzer Agent, dan Fuzzing Agent. Agen pembuat kode menerima masukan dari pemeriksaan keamanan, kemudian memperbaiki hasilnya. Pada metode versi pertama, Static Analyzer Agent adalah LLM yang diarahkan untuk mencari

kelemahan berdasarkan CWE. Bandit digunakan untuk menilai hasil percobaan, sehingga perannya berbeda dari reviewer LLM dalam sistem tersebut.

Dengan GPT-4o, jumlah kode rentan pada SecurityEval turun dari 59/121 menjadi 44/121. Selisihnya sekitar 12,4 poin persentase berdasarkan jumlah sampel. Angka penurunan 13% dalam paper tidak digunakan sebagai penurunan relatif. Sementara itu, skor pass@1 pada HumanEval turun dari 0,9085 menjadi 0,8780 (Nunez et al., 2024). Hasil ini menunjukkan adanya perbaikan keamanan yang disertai sedikit penurunan keberhasilan fungsi. Pengujian fuzzing memeriksa kegagalan saat kode dijalankan, tetapi tidak semua kelemahan keamanan menimbulkan kegagalan yang sama.

Multi-LLMSecCodeEval membandingkan sepuluh alur yang mencakup pembuatan kode, deteksi kerentanan, dan perbaikan. Pada SecLLMEval serta SecLLMHolmes, penggabungan beberapa model, CodeQL, dan pemeriksaan lintas-model dilaporkan memberikan hasil yang lebih konsisten daripada kerja sama LLM tanpa alat statis (Sabir et al., 2026). Penelitian tersebut sudah membahas alur dari pembuatan hingga perbaikan kode. Oleh karena itu, alur tersebut tidak dijadikan kebaruan tersendiri dalam penelitian ini. Hasil untuk kelompok model besar dan percobaan model lebih kecil dibahas sebagai konfigurasi berbeda, sehingga persentasenya tidak digabungkan menjadi satu hasil.

Hasil tersebut juga menunjukkan perlunya memisahkan manfaat alat dari manfaat kerja sama agen. Jika pemindai dan beberapa reviewer ditambahkan sekaligus, peningkatan hasil belum tentu berasal dari diskusi antaragen. Penelitian ini karena itu merencanakan pembanding berupa LLM tunggal, SAST saja, serta SAST dengan LLM. Tugas, jumlah kesempatan perbaikan, dan anggaran penggunaan model perlu dibuat sebanding atau dijelaskan perbedaannya.

2.3.3 Penggabungan hasil pemeriksaan

Salah satu cara menggabungkan hasil adalah voting mayoritas, yaitu memilih keputusan yang paling banyak diberikan agen. Cara ini mudah diterapkan, tetapi dapat menghasilkan keputusan salah jika sebagian besar agen memiliki

kesalahan yang sama. Cara lain adalah debat, yaitu meminta agen membahas perbedaan pendapat sebelum mengambil kesimpulan.

ReasonVul menggunakan tiga cara penalaran, yaitu deduktif, induktif, dan abduktif. Secara sederhana, agen memeriksa kode berdasarkan aturan umum, pola dari contoh, dan dugaan penyebab masalah. Ketiga tugas tersebut menggunakan Phi4, Llama4, dan DeepSeek. Ketika hasilnya berbeda, agen melakukan debat. Pada analisis konflik dalam percobaan PrimeVul, 389 dari 542 konflik diselesaikan dengan benar. Pada JITVUL, sistem memperoleh PairAcc 28,67% (Peng et al., 2026). PairAcc menghitung keberhasilan mengenali kode rentan dan versi perbaikannya secara benar sebagai satu pasangan. Ukuran ini berbeda dari akurasi untuk satu potongan kode.

VulTrial menggunakan pembagian peran seperti ruang sidang. Peneliti keamanan mengajukan dugaan kelemahan, penulis kode memberikan penjelasan, lalu moderator dan review board menilai pembahasannya (Widyasari et al., 2025). MAVUL menggunakan analis konteks dan arsitek keamanan dengan GPT-4o. Agen evaluasinya memakai label acuan untuk menilai hasil eksperimen (Li et al., 2025b). Agen tersebut berbeda dari arbiter saat penggunaan gate, yang harus mengambil keputusan tanpa mengetahui label jawaban. Kedua pendekatan memberi contoh cara mempertanyakan temuan sebelum menerimanya.

Debat tidak selalu menghasilkan keputusan yang benar. Refute-or-Promote menggunakan pemeriksaan bertahap, kritik dari model lain, dan pengujian nyata untuk menolak temuan yang keliru. Laporan kasusnya menunjukkan bahwa kesepakatan beberapa reviewer masih dapat dibantah oleh hasil pengujian (Agarwal, 2026). Wynn dkk. juga menemukan bahwa agen dapat beralih dari jawaban benar ke jawaban salah setelah menerima alasan dari agen lain (Wynn et al., 2025). Karena itu, persetujuan banyak agen tidak menggantikan kebutuhan akan bukti.

2.3.4 Arbiter dan biaya penggunaan model

Arbiter adalah bagian yang menentukan keputusan akhir berdasarkan hasil pemeriksaan. Dalam rancangan penelitian ini, arbiter menggunakan aturan non-LLM yang disimpan dalam YAML. Agen memberikan temuan dan penjelasan,

sedangkan arbiter menentukan *pass*, *revise*, atau *block*. Keputusan pass berarti kode diterima berdasarkan pemeriksaan yang ditetapkan. Revise berarti kode perlu diperbaiki, sedangkan block berarti kode perlu ditinjau manusia. Temuan berkeparahan tinggi yang belum selesai ditangani mencegah kode memperoleh pass.

Kode yang diterima dapat diberi label *Verified*, dengan arti telah melewati pemeriksaan yang ditetapkan. Label tersebut bukan jaminan bahwa kode bebas dari seluruh kelemahan. Alasan keputusan perlu mencatat aturan, versi kode, dan hasil pengujian. Respons agen yang tidak lengkap serta pengujian yang gagal dijalankan perlu dipisahkan dari hasil yang memenuhi seluruh syarat penerimaan.

Penggunaan beberapa model juga menambah biaya. Wang menggabungkan model layanan dengan pemeriksa lokal untuk menyeimbangkan hasil deteksi dan biaya (Wang, 2026). Pada gate, biaya tidak hanya berasal dari pembuatan kode. Pemeriksaan, diskusi, pembuatan pengujian, dan perbaikan berulang turut memakai token serta waktu. Menjalankan beberapa agen bersamaan dapat memperpendek waktu tunggu, tetapi tidak selalu mengurangi jumlah penggunaan model.

2.4 Perbaikan Kode Otomatis

2.4.1 Perbaikan berdasarkan hasil pemeriksaan

Perbaikan kode otomatis bertujuan menghilangkan kelemahan tanpa mengubah fungsi yang dibutuhkan pengguna. Sebuah perubahan belum dapat dianggap berhasil hanya karena peringatan alat hilang. Model dapat saja menghapus bagian kode yang bermasalah, tetapi sekaligus menghilangkan fungsi program. Oleh karena itu, kode yang telah diperbaiki perlu diperiksa kembali dari sisi keamanan dan kebenaran hasilnya.

Salah satu cara memperbaiki kode adalah memberikan hasil analisis statis kepada LLM. Feedback-Driven Security Patching menggunakan keluaran Bandit untuk mengarahkan perbaikan kode Python (Alrashedy et al., 2024). Static Analysis as a Feedback Loop juga menggunakan Bandit dan Pylint untuk memperbaiki beberapa aspek kualitas kode (Blyth et al., 2025). Pada kedua pendekatan tersebut, model memperoleh informasi tentang masalah yang ditemukan alat. Informasi ini

membantu model melakukan perbaikan yang lebih terarah daripada hanya diminta memeriksa jawabannya sendiri.

Penggunaan beberapa sumber masukan dibahas dalam penelitian Sriram dkk. Model menerima hasil kompilasi GCC, pemeriksaan CodeQL, dan pengujian dengan KLEE. Pada 1.720 program C/C++ hasil CodeLlama-7B, tingkat cacat keamanan menurut CodeQL turun dari 58,55% menjadi 22,19%. Percobaan lainnya menggunakan 1.522 program hasil DeepSeek-Coder-1.3B (Sriram et al., 2026). Hasil ini mendukung penggunaan alat untuk membantu perbaikan, tetapi alat pengujian C/C++ tersebut tidak dapat langsung digunakan pada potongan kode Python.

LLMLOOP menghubungkan perbaikan kode dengan perbaikan pengujiannya melalui beberapa tahap umpan balik (Ravi et al., 2025). Pengujian mempunyai peran penting karena membantu memeriksa apakah perubahan kode tetap memenuhi tugas semula. Hilangnya peringatan pada satu alat tidak cukup untuk menjelaskan seluruh hasil perbaikan. Alat yang digunakan, jenis kelemahan yang diperiksa, dan pengujian fungsi perlu dijelaskan bersama.

2.4.2 Pemisahan agen perbaikan

Model yang membuat kode dapat diminta memperbaiki keluarannya sendiri. Cara ini disebut *self-repair*. Pilihan lainnya adalah menggunakan agen Fixer yang terpisah dari generator. Model lain dapat memberikan cara penyelesaian berbeda, tetapi tetap mungkin memiliki kesalahan yang sama. Olausson dkk. menunjukkan bahwa manfaat self-repair bergantung pada kualitas umpan balik dan biaya percobaan (Olausson et al., 2024). Oleh karena itu, penggunaan model berbeda perlu dibandingkan dengan model yang sama pada tugas dan anggaran yang sebanding.

Model juga dapat menerima contoh perbaikan dari penyimpanan pengetahuan atau *knowledge store*. Pengambilan contoh yang sesuai disebut *retrieval*. AutoPatch menggunakan informasi CVE untuk membantu pemeriksaan dan perbaikan, sedangkan penelitian multi-tool menyimpan hasil perbaikan yang telah lolos pengujian agar dapat digunakan kembali (Seo et al., 2025; Sriram et al., 2026).

Penyimpanan contoh perbaikan memerlukan pemisahan antara contoh yang boleh diambil model dan sampel untuk penilaian akhir. Jika patch sampel uji yang sama tersedia dalam retrieval, hasil pengujian tidak lagi menunjukkan kemampuan pada kasus yang belum diketahui. Setiap contoh juga perlu dikaitkan dengan versi kode dan bukti pengujian. Keberhasilan pada satu kasus belum menjamin bahwa patch tersebut sesuai untuk kasus lain.

2.4.3 Pemeriksaan ulang dan batas perbaikan

Perbaikan dapat menimbulkan masalah baru pada bagian yang sebelumnya tidak bermasalah. Keadaan ini disebut regresi. Studi pada perbaikan Infrastructure-as-Code menunjukkan bahwa hasil pemeriksaan keamanan dapat memburuk setelah perubahan kode. Penulis juga membedakan regresi yang benar-benar terjadi dari perubahan hasil ukur akibat penataan ulang sumber daya (Agyekum & Santos, 2026). Temuan pada Terraform tersebut tidak langsung menunjukkan tingkat risiko pada Python, tetapi menjelaskan alasan perlunya pemeriksaan ulang setelah perubahan.

Patch perlu melalui kembali pemeriksaan yang berlaku. Pengujian yang sebelumnya menemukan masalah dapat dipertahankan untuk memeriksa apakah masalah itu muncul kembali. Pengujian fungsi juga diperlukan agar model tidak menghilangkan tugas program saat memperbaiki kelemahan. Untuk fungsi atau potongan kode, pengujian dapat membutuhkan program pembantu atau *test harness* yang menyiapkan masukan dan memeriksa hasil. CodeSecEval memberi contoh penyediaan pengujian fungsi dan keamanan, sedangkan jalur dinamis QASecClaw tidak dijalankan pada evaluasi statisnya (Ameen et al., 2026; Wang et al., 2026a). Dengan demikian, kemampuan eksekusi perlu diperiksa untuk setiap tugas.

Jumlah perbaikan perlu dibatasi agar proses tidak terus mengulang perubahan yang tidak menyelesaikan masalah. Jika batas tercapai, kode tidak langsung diterima. Arbiter menentukan tindak lanjut sesuai masalah yang masih tersisa. Begitu pula, kode yang belum dapat dijalankan tidak boleh dianggap telah lulus pengujian dinamis hanya karena tidak ada laporan kegagalan.

2.5 Ketahanan Gate terhadap Serangan

2.5.1 Serangan pada agen pemeriksa

Agen pemeriksa atau *reviewer* LLM membaca kode dan informasi pendukung untuk menilai keamanan. Informasi tersebut dapat berasal dari pihak yang tidak tepercaya. Penyerang dapat menyisipkan instruksi dalam komentar, docstring, atau teks lain agar model mengabaikan kelemahan. Upaya mengalihkan model melalui instruksi semacam ini disebut *prompt injection*. Pada gate, serangan dapat bertujuan menyembunyikan temuan, menurunkan tingkat keparahan, atau membuat kode rentan memperoleh keputusan pass.

Serangan perlu dibedakan berdasarkan tugas model yang disasar. TAPI menggunakan pemicu dalam komentar untuk memengaruhi kode yang dibuat model (Yang et al., 2024). Your AI, My Shell membahas pengalihan editor agentik melalui sumber pengembangan yang telah disisipi instruksi penyerang (Liu et al., 2026). Kedua penelitian tersebut menunjukkan risiko dari masukan tidak tepercaya, tetapi tugasnya berbeda dari pemeriksaan keamanan kode. Hasil serangan pada generator atau editor tidak langsung menunjukkan tingkat keberhasilan serangan pada reviewer.

Batas kemampuan penyerang memengaruhi hasil pengujian. Penyerang yang hanya dapat mengubah komentar memiliki akses berbeda dari penyerang yang dapat mengubah perintah program, keluaran alat, atau isi knowledge store. Penelitian prompt injection membedakan instruksi pengembang dari masukan tidak tepercaya yang dibaca model (Greshake et al., 2023; Liu et al., 2024). Oleh karena itu, vektor serangan dan akses penyerang perlu dinyatakan sebelum hasil ketahanan dibandingkan.

2.5.2 Hasil penelitian serangan pada reviewer

Maloyan dan Namiot menunjukkan bahwa penilaian LLM dapat dipengaruhi oleh prompt injection. Mereka membahas penggunaan beberapa model penilai dan penilaian komparatif sebagai arah pertahanan (Maloyan & Namiot, 2025). Namun, tugas penilaian umum tidak sama dengan pencarian kelemahan kode. Beberapa model juga dapat membaca instruksi penyerang yang sama. Oleh karena itu,

penambahan reviewer tetap perlu diuji dan belum dapat dianggap cukup untuk mencegah serangan.

Hasil yang berbeda ditemukan dalam penelitian Thornton. Percobaan menggunakan 100 sampel kode rentan dari Python, JavaScript, dan Java yang dinilai oleh delapan model komersial maupun terbuka. Pengaruh komentar serangan tidak signifikan secara statistik pada pengujian berpasangan. Penghapusan komentar justru menurunkan kemampuan deteksi model yang lebih lemah. Sebaliknya, pemberian temuan SAST sebagai petunjuk membantu pemeriksaan (Thornton, 2026). Hasil tersebut berlaku pada sampel dan jenis komentar yang diuji, sehingga tidak berarti reviewer kebal terhadap seluruh serangan.

Perbedaan hasil ini menunjukkan pentingnya mempertahankan tugas utama ketika menilai pertahanan. Sistem perlu tetap mampu membaca konteks yang dibutuhkan untuk memeriksa kode. Tinjauan ancaman agen juga menekankan bahwa perlindungan, kegunaan, dan waktu pemrosesan perlu dinilai bersama (Wang et al., 2026b). Pertahanan yang membuat model mengabaikan semua informasi tambahan dapat mengurangi serangan, tetapi juga dapat mengurangi kemampuannya menyelesaikan tugas.

2.5.3 Sanitizer dan aturan keputusan

Sanitizer adalah bagian non-LLM yang menyiapkan masukan sebelum dibaca reviewer, misalnya dengan mengurangi komentar yang mengandung instruksi pengalih. Penghapusan informasi belum tentu membantu, sebagaimana ditemukan Thornton pada model yang lebih lemah (Thornton, 2026). Pada Python, docstring juga dapat diakses melalui atribut `__doc__` saat program berjalan (Python Software Foundation, n.d.). Karena itu, penghapusan docstring tidak dapat selalu dianggap mempertahankan perilaku program. Proyeksi untuk reviewer perlu dibedakan dari kode asli yang dipindai dan dijalankan, serta tetap menyimpan hubungan nomor baris.

Manfaat sanitizer akan dinilai dengan membandingkan gate yang menggunakannya dan gate yang tidak menggunakannya. Pengujian memakai sampel yang sama, baik tanpa serangan maupun setelah disisipi serangan.

Kelemahan kode harus tetap ada pada kedua sampel. Jika perubahan komentar sekaligus menghilangkan kelemahan atau mengubah fungsi program, perbedaan keputusan reviewer tidak lagi menunjukkan pengaruh serangan pada kasus yang sama.

Selain keberhasilan serangan, pengujian perlu mencatat perubahan presisi, recall, dan jumlah kode rentan yang diterima. *Attack success rate* adalah proporsi serangan yang mencapai tujuan pada kasus yang diuji. Perhitungannya perlu menjelaskan kasus mana yang masuk ke penyebut. Kelemahan yang memang tidak terdeteksi sejak awal perlu dibedakan dari kelemahan yang baru terlewat setelah komentar serangan ditambahkan.

Aturan arbiter dapat membatasi keputusan reviewer, tetapi serangan juga dapat berada di luar komentar. PhantomSkill menunjukkan bahwa perilaku berbahaya dapat disamarkan dalam kode yang tampak memiliki kelemahan biasa (Lin & Yu, 2026). Karena itu, ruang lingkup perlindungan perlu menyebutkan masukan yang diperiksa dan komponen yang dapat diubah penyerang. Hasil pertahanan komentar tidak langsung berlaku untuk aturan gate, keluaran alat, atau penyimpanan pengetahuan.

DAFTAR PUSTAKA

Agarwal, A. (2026). *Refute-or-Promote: An Adversarial Stage-Gated Multi-Agent Review Methodology for High-Precision LLM-Assisted Defect Discovery*. https://arxiv.org/abs/2604.19049

Agrawal, V., & Ahi, K. (2025). *LLM-Driven SAST-Genius: A Hybrid Static Analysis Framework for Comprehensive and Actionable Security*. https://arxiv.org/abs/2509.15433

Agyekum, B., & Santos, F. (2026). Does Fixing Break Security? An Empirical Study of Security Degradation in Iterative LLM-Driven Infrastructure-As-Code Repair. *20th International Symposium on Empirical Software Engineering and Measurement (ESEM 2026), Leibniz International Proceedings in Informatics*, *394*, 50:1–50:20. https://doi.org/10.4230/LIPIcs.ESEM.2026.50

Alrashedy, K., Aljasser, A., Tambwekar, P., & Gombolay, M. (2024). *Can LLMs Patch Security Issues?*. https://arxiv.org/abs/2312.00024

Ameen, M. R., Alam, M. T. U., & Islam, A. (2026). *QASecClaw: A Multi-Agent LLM Approach for False Positive Reduction in Static Application Security Testing*. https://arxiv.org/abs/2605.01885

Belozerov, V., Barclay, P. J., & Sami, A. (2026). Secure Coding with AI – From Detection to Repair. *Empirical Software Engineering*, *31*. https://doi.org/10.1007/s10664-026-10812-8

Bhatt, M., Chennabasappa, S., Nikolaidis, C., Wan, S., Evtimov, I., Gabi, D., Song, D., Ahmad, F., Aschermann, C., Fontana, L., Frolov, S., Giri, R. P., Kapil, D., Kozyrakis, Y., LeBlanc, D., Milazzo, J., Straumann, A., Synnaeve, G., Vontimitta, V., … Saxe, J. (2023). *Purple Llama CyberSecEval: A Secure Coding Benchmark for Language Models*. https://arxiv.org/abs/2312.04724v1

Blyth, S., Licorish, S. A., Treude, C., & Wagner, M. (2025). *Static Analysis as a Feedback Loop: Enhancing LLM-Generated Code Beyond Correctness*. https://arxiv.org/abs/2508.14419

Chen, Y., Sun, W., Fang, C., Chen, Z., Ge, Y., Han, T., Zhang, Q., Liu, Y., Chen, Z., & Xu, B. (2025). *Security of Language Models for Code: A Systematic Literature Review*. https://arxiv.org/abs/2410.15631

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). *Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection*. https://arxiv.org/abs/2302.12173

Hariharan, S., Majid, Z. A., Veuthey, J. R., & Haimes, J. (2024). *Rethinking CyberSecEval: An LLM-Aided Approach to Evaluation Critique*. https://arxiv.org/abs/2411.08813

Iranmanesh, M., Sabet, S. M., Marefat, S., Ghasr, A. J., Wilson, A., Sharafaldin, I., & Tayebi, M. A. (2025). *ZeroFalse: Improving Precision in Static Analysis with LLMs*. https://arxiv.org/abs/2510.02534v1

Khanmohammadi, K., Roy, P., Khoury, R., Hamou-Lhadj, A., Konan, W. P., Da Re, A., & Melo, N. R. (2026). *WildCode Revisited: A Comprehensive Empirical Study on the Security of LLM-Generated Code*. https://arxiv.org/abs/2512.04259

Kharma, M., Sabbah, A., Alkhanafseh, M., Hammoudeh, M., & Mohaisen, D. (2026). *An Empirical Evaluation of LLM-Generated Code Security Across Prompting Methods*. https://arxiv.org/abs/2605.24298

Li, Y., Joshi, K., Wang, X., & Wong, E. (2025b). *MAVUL: Multi-Agent Vulnerability Detection via Contextual Reasoning and Interactive Refinement*. https://arxiv.org/abs/2510.00317v1

Li, Z., Dutta, S., & Naik, M. (2025a,). IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities. *International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/582d4e27fa24168f3af1f4582655034b-Abstract-Conference.html

Lin, Y.-T., & Yu, C.-M. (2026). *PhantomSkill: Malicious Code Injection in Agent Skill Ecosystems*. https://arxiv.org/abs/2606.19191

Liu, Y., Zhao, Y., Lyu, Y., Zhang, T., Wang, H., & Lo, D. (2026). *"Your AI, My Shell": Demystifying Prompt Injection Attacks on Agentic AI Coding Editors*. https://arxiv.org/abs/2509.22040

Liu, Y., Jia, Y., Geng, R., Jia, J., & Gong, N. Z. (2024). Formalizing and Benchmarking Prompt Injection Attacks and Defenses. *33rd USENIX Security Symposium (USENIX Security 24)*, 1831–1847. https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

Maloyan, N., & Namiot, D. (2025). *Adversarial Attacks on LLM-as-a-Judge Systems: Insights from Prompt Injections*. https://arxiv.org/abs/2504.18333

Morkonda, S. G., Selim, M., & Assal, H. (2026). *Security of LLM-generated Code: A Comparative Analysis*. https://arxiv.org/abs/2605.23091

Nunez, A., Islam, N. T., Jha, S., & Rad, P. (2024). *AutoSafeCoder: A Multi-Agent Framework for Securing LLM Code Generation through Static Analysis and Fuzz Testing*. https://arxiv.org/abs/2409.10737v1

Olausson, T. X., Inala, J. P., Wang, C., Gao, J., & Solar-Lezama, A. (2024). *Is Self-Repair a Silver Bullet for Code Generation?*. https://arxiv.org/abs/2306.09896

Peng, X., Lin, B., Wang, J., Li, X., Ma, J., Yu, J., Mao, X., & Wang, S. (2026). Three Heads Are Better Than One: A Multi-perspective Reasoning Framework for Enhanced Vulnerability Detection. *Proceedings of the ACM on Software Engineering*, *3*(FSE), 4323–4346. https://doi.org/10.1145/3808197

Python Software Foundation. (n.d.). *Data model: Callable types / user-defined functions*. Retrieved October 20, 2026, from https://docs.python.org/3.11/reference/datamodel.html#user-defined-functions

Ravi, R., Bradshaw, D., Ruberto, S., Jahangirova, G., & Terragni, V. (2025). LLMLOOP: Improving LLM-Generated Code and Tests through Automated Iterative Feedback Loops. *2025 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 930–934. https://doi.org/10.1109/ICSME64153.2025.00109

Sabir, B., Liu, S., Jang, S. I., Abuadbba, S., Gao, Y., Moore, K., Kim, S., Kim, H., & Nepal, S. (2026). *Does Teaming-Up LLMs Improve Secure Code Generation? A Comprehensive Evaluation with Multi-LLMSecCodeEval*. https://arxiv.org/abs/2603.22717

Seo, M., Choi, W., Shin, S., & You, M. (2025). *AutoPatch: Multi-Agent Framework for Patching Real-World CVEs Generated by Outdated LLMs*. https://arxiv.org/abs/2505.04195

Shahid, M. U., Ahmed, C. M., & Ranjan, R. (2025). *LLM-CSEC: Empirical Evaluation of Security in C/C++ Code Generated by Large Language Models*. https://arxiv.org/abs/2511.18966v1

Siddiq, M. L., & Santos, J. C. S. (2022). SecurityEval Dataset: Mining Vulnerability Examples to Evaluate Machine Learning-Based Code Generation Techniques. *Proceedings of the 1st International Workshop on Mining Software Repositories Applications for Privacy and Security*, 29–33. https://doi.org/10.1145/3549035.3561184

Sriram, V., Pandita, S., Lakshmanan, A., Shamraj, A., & Saha, S. (2026). *Improving LLM-Assisted Secure Code Generation through Retrieval-Augmented-Generation and Multi-Tool Feedback*. https://arxiv.org/abs/2601.00509v1

Tang, X., Kim, K., Song, Y., Lothritz, C., Li, B., Ezzini, S., Tian, H., Klein, J., & Bissyandé, T. F. (2024). CodeAgent: Autonomous Communicative Agents for Code Review. *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 11279–11313. https://doi.org/10.18653/v1/2024.emnlp-main.632

Thornton, S. (2026). *Can Adversarial Code Comments Fool AI Security Reviewers – Large-Scale Empirical Study of Comment-Based Attacks and Defenses Against LLM Code Analysis*. https://arxiv.org/abs/2602.16741

Tony, C., Mutas, M., Ferreyra, N. E. D., & Scandariato, R. (2023). *LLMSecEval: A Dataset of Natural Language Prompts for Security Evaluations*. https://arxiv.org/abs/2303.09384v1

Tsigkourakos, G., & Patsakis, C. (2026). *QRS: A Rule-Synthesizing Neuro-Symbolic Triad for Autonomous Vulnerability Discovery*. https://arxiv.org/abs/2602.09774v1

Wang, J., Cao, L., Luo, X., Cao, Y., Li, Z., Xiao, Y., Zhao, M., Jatowt, A., & Cai, Y. (2026a). *Toward Secure Code Generation: Bridging Correctness and Security via Task-Adaptive Vulnerability Modeling and Execution-Based Benchmarking*. https://arxiv.org/abs/2407.02395v3

Wang, J., Luo, X., Cao, L., He, H., Huang, H., Xie, J., Jatowt, A., & Cai, Y. (2024). *Is Your AI-Generated Code Really Safe? Evaluating Large Language Models on Secure Code Generation with CodeSecEval*. https://arxiv.org/abs/2407.02395v2

Wang, P., Li, X., Xiang, C., Zhang, J., Li, Y., Zhang, L., Wang, X., & Tian, Y. (2026b). *The Landscape of Prompt Injection Threats in LLM Agents: From Taxonomy to Analysis*. https://arxiv.org/abs/2602.10453

Wang, Z. G. (2026). *Strategic Heterogeneous Multi-Agent Architecture for Cost-Effective Code Vulnerability Detection*. https://arxiv.org/abs/2604.21282

Widyasari, R., Weyssow, M., Irsan, I. C., Ang, H. W., Liauw, F., Ouh, E. L., Shar, L. K., Kang, H. J., & Lo, D. (2025). *Let the Trial Begin: A Mock-Court Approach to Vulnerability Detection using LLM-Based Agents*. https://arxiv.org/abs/2505.10961v2

Wynn, A., Satija, H., & Hadfield, G. (2025). *Talk Isn't Always Cheap: Understanding Failure Modes in Multi-Agent Debate*. https://arxiv.org/abs/2509.05396

Yang, Y., Yao, H., Yang, B., He, Y., Li, Y., Zhang, T., Qin, Z., & Ren, K. (2024). *TAPI: Towards Target-Specific and Adversarial Prompt Injection against Code LLMs*. https://arxiv.org/abs/2407.09164v3